

中国科学院大学计算机组成原理实验报告

|      |                      |      |                 |
|------|----------------------|------|-----------------|
| 姓名   | 周禹驰                  | 学号   | 2024K8009929027 |
| 专业   | 计算机科学与技术             | 实验编号 | Prj4-选做 3       |
| 实验名称 | 复杂 IO 外设:DNN 硬件加速器协同 |      |                 |

完整实验流程包括设计、开发、调试三个步骤：

- **设计**步骤为在代码编写前的准备阶段,主要包括 DNN 加速器地址映射、MMIO 控制流程、CPU/Cache/DDR 协同方式和性能评价方法设计。
- **开发**步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写,启用 DNN 加速器配置,保证 CPU 可通过 MMIO 启动硬件任务并轮询完成状态。
- **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证,重点检查 DNN 共享 DDR 区域一致性、MMIO 旁路和 custom\_bit\_gen 实现结果。

一、设计成果(对设计流程的总结。需附上设计结构图,可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF,再插入到 L<sup>A</sup>T<sub>E</sub>X 中)

1 总体设计目标

本实验对应讲义中“复杂 IO 外设”的 DNN 硬件加速器部分。设计目标是在 SoC 中启用 DNN 加速器,使 RISC-V32 处理器通过 MMIO 寄存器启动硬件卷积/池化任务,并通过共享 DDR 区域传递输入、权重、输出和参考结果。当前工程使用课程提供的 dnn\_acc.dcp, CPU 与 Cache 负责正确访问控制寄存器、保持共享数据一致性并输出性能计数器。

2 DNN 数据与控制地址

软件侧 conv.c 中定义了固定数据地址和控制寄存器地址。硬件加速路径中, CPU 写 start 寄存器启动 DNN,随后轮询 done 寄存器最低位;DNN 加速器作为 AXI master 访问 DDR 中的输入、权重和输出区域。

| 用途         | 地址          | 说明               |
|------------|-------------|------------------|
| 权重 weight  | 0x0800_0000 | 卷积核和 bias 数据     |
| 输入 rd      | 0x080d_8f90 | 输入图像数据           |
| 输出 wr      | 0x080d_9690 | 硬件或软件写回结果        |
| 参考结果       | 0x0700_0000 | golden/result 区域 |
| start MMIO | 0x6003_0000 | 写 1 启动 DNN 加速器   |
| done MMIO  | 0x6003_0008 | 轮询 bit0 判断完成     |

表 1: DNN 数据与控制地址

3 系统协同结构

DNN 加速器与 CPU 共享 DDR,但二者并不共享 D-cache 内部状态。因此,凡是 CPU 和 DNN 都会访问的数据区,都不能只停留在 CPU 的 D-cache 中。系统结构如图 1所示。

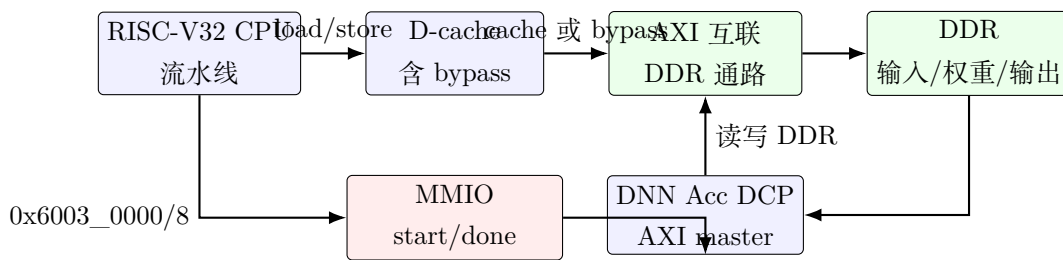


图 1: CPU、D-cache 与 DNN 加速器协同结构

#### 4 软件算法与硬件加速路径

软件卷积输入为  $1 \times 1 \times 28 \times 28$ , 权重为  $20 \times 1 \times 5 \times 5$ , 卷积输出为  $1 \times 20 \times 24 \times 24$ , 随后进行  $2 \times 2$  池化得到  $1 \times 20 \times 12 \times 12$  结果。若定义 `USE_HW_ACCEL`, 软件不再执行完整嵌套循环, 而是启动硬件加速器并等待完成。

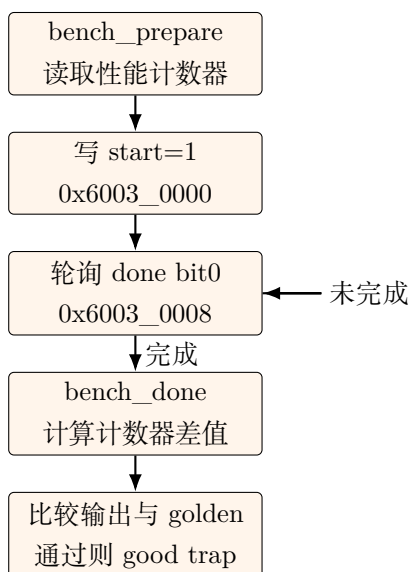


图 2: DNN 硬件加速控制流程

## 二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

### 1 工程配置

工程通过 `arch_options.tcl` 启用 I-cache、D-cache 和 DNN 加速器。配置中 `dnn_acc=1` 表示使用课程提供的 DNN 加速器 DCP; CPU 侧不直接实现卷积阵列, 而是负责软件控制、Cache 一致性和性能统计。

Listing 1: DNN 相关工程配置摘要

```
set cpu_freq 100
set icache 1
set dcache 1
set simple_dma 0
set dnn_acc 1
```

## 2 MMIO 访问与 D-cache 旁路

start 和 done 寄存器位于 0x6003\_0000 地址空间,必须作为 I/O 访问处理。如果该区域进入 D-cache,写 start 可能无法到达 DNN 外设,读 done 也可能反复命中旧值,导致软件永久等待。因此 D-cache 对 addr >= 0x4000\_0000 的访问全部旁路。

Listing 2: MMIO 和 DNN 共享区旁路条件摘要

```
assign bypass_mmio = addr >= 32'h4000_0000;
assign bypass_dnn = (addr >= 32'h0700_0000) &&
                    (addr < 32'h0810_0000);
assign bypass = bypass_mmio | bypass_dnn | low_addr_special;
```

## 3 共享 DDR 一致性

DNN 输入、权重、输出和参考结果位于 0x0700\_0000 到 0x0810\_0000 附近。若 CPU 写入的数据停留在 D-cache 脏块中,DNN 从 DDR 读取时会得到旧值;若 DNN 已经把输出写回 DDR,而 CPU 从 D-cache 命中旧输出块,结果比对也会失败。因此 DNN 共享数据区也采用旁路访问,保证 CPU 和 DNN 看到的是同一份 DDR 数据。

| 问题          | 现象                 | 处理方法      |
|-------------|--------------------|-----------|
| start 写入被缓存 | DNN 不启动,done 一直为 0 | MMIO 区域旁路 |
| done 读取被缓存  | CPU 永久轮询旧值         | MMIO 区域旁路 |
| 输入/权重脏块未写回  | DNN 计算错误或超时        | DNN 共享区旁路 |
| 输出旧块命中      | CPU 比较结果错误         | DNN 共享区旁路 |

表 2: DNN 协同中的一致性风险

## 4 性能评价实现

CPU 保留 cycle、retire 和 mem\_req 三个计数器。软件在 bench\_prepare 记录初值,在 bench\_done 记录差值。硬件加速版本的预期特征是 retire 指令数显著少于纯软件卷积/池化版本,总 cycles 也应下降;若 cycles 没有下降,应进一步检查 DNN 是否真实启动、done 是否真实返回、custom\_bit\_gen 是否存在布局布线超时。

| 实现方式        | cycles        | inst_retired | mem_req | FPGA 端到端时间 |
|-------------|---------------|--------------|---------|------------|
| sw_conv     | 1,002,279,691 | 233,830,415  | 793,917 | 6699.42 ms |
| sw_conv_mul | 82,093,572    | 6,042,774    | 652,730 | 564.17 ms  |
| hw_conv     | 1,061,257     | 130,948      | 43,558  | 33.13 ms   |

表 3: DNN 软件与硬件加速性能对比

$$Speedup = \frac{C_{software}}{C_{hardware}}$$

从 Table 3 可以看到,三组测试均在 FPGA 上通过校验并进入 Hit good trap,说明硬件加速路径和软件路径得到一致结果。以 cycle 计,hw\_conv 相对纯软件 sw\_conv 从 1,002,279,691 周期降至 1,061,257 周期,加速约 944.43 倍;相对乘法优化后的 sw\_conv\_mul 也从 82,093,572 周期降至 1,061,257 周期,加速约 77.36 倍。若按 GitLab 日志中的端到端运行时间计算,硬件版本相对 sw\_conv 从 6699.42 ms 降至 33.13 ms,加速约 202.22 倍;相对 sw\_conv\_mul 从 564.17 ms 降至 33.13 ms,加速约 17.03 倍。端到端时间加速比低于 cycle 加速比,主要是因为 FPGA 配置、程序装载、host 交互和测试框架开销不会随 DNN 核心计算等比例下降。

加速的根本原因是计算形态发生了变化。纯软件 `sw_conv` 需要 CPU 逐条执行卷积和池化的多层循环, 包含大量乘法、加法、地址计算、循环分支和 Load/Store, 因此提交指令数达到 233,830,415 条。`sw_conv_mul` 通过软件侧乘法优化把提交指令数降到 6,042,774 条, 但卷积计算仍由 CPU 串行发射和提交。硬件版本则由 CPU 完成少量数据准备、MMIO 启动和 done 轮询, 卷积/池化主体交给 DNN 加速器内部并行数据通路完成, 因此 CPU 提交指令数进一步降到 130,948 条, 约为纯软件的 1/1785.67。

访存行为也得到明显改善。硬件版本的 `mem_req` 为 43,558 次, 相比 `sw_conv` 减少约 18.23 倍, 相比 `sw_conv_mul` 减少约 14.99 倍。软件卷积中, CPU 需要反复从 DDR 或 Cache 路径读取输入、权重、中间结果并写回输出; 硬件加速器则通过 AXI master 按更规整的访问模式搬运数据, 并在内部复用输入窗口和权重数据, 减少 CPU 侧访存请求和流水线 MEM 级阻塞。也就是说, 本实验中的 DNN 加速不是单纯“减少几条指令”, 而是把高重复度卷积运算从顺序 CPU 执行迁移到专用硬件数据通路, 同时减少 CPU 取指、循环控制和访存等待开销。

### 三、 学习总结(总结设计、开发与调试过程中遇到的问题, 及实验中的收获与不足)

#### 1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

| 序号 | 问题类别     | 出错模块           | 问题现象                  | 产生原因                          | 解决办法                                 |
|----|----------|----------------|-----------------------|-------------------------------|--------------------------------------|
| 1  | 系统一致性    | D-cache/DNN    | dnn_test 超时或 bad trap | DNN 共享 DDR 数据被 CPU D-cache 缓存 | 对 0x0700_0000 到 0x0810_0000 区域旁路     |
| 2  | 复杂 IO 访问 | MMIO           | 写 start 后 done 一直为 0  | start/done 寄存器没有真实访问外设        | 对 0x4000_0000 以上 I/O 空间旁路            |
| 3  | FPGA 实现  | custom_bit_gen | 布局布线拥塞或超时             | DNN、Cache、AXI 互联资源和时序压力较大     | 降低频率、减少额外逻辑或检查 AXI register slice 约束 |

表 4: DNN 实验问题总结

- **DNN 问题补充。**DNN 实验的难点不只在计算本身, 而在 CPU、Cache、AXI、DDR 和外设之间的数据可见性。若只从 CPU 指令执行角度观察, 很容易误判为流水线死锁; 实际原因可能是 done 读值被缓存, 或者 DNN 读取到旧输入数据。

- **其他关键模块问题补充。**`custom_bit_gen` 失败时应区分功能错误和实现拥塞。若错误日志出现在 place 或 route 阶段, 并提示 congestion 或 timeout, 通常说明 FPGA 资源/布线压力过大, 不等同于 RTL 行为仿真错误。

#### 2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

### 四、 扩展思考(对讲义中思考题(如有)的理解和回答)

- **思考题 1: 为什么硬件加速器接入后, Cache 正确性比纯 CPU 程序更敏感?**

纯 CPU 程序中, D-cache 内部脏块最终仍由同一个 CPU 读取和写回, 很多一致性问题不会暴露。但 DNN 加速器绕过 CPU D-cache 直接访问 DDR, 如果 CPU 写入的数据还没有到达 DDR, 或者 DNN 写回的数据被 CPU 旧 cache line 遮蔽, 就会出现 CPU 和 DNN 看到不同数据版本的问题。

- **思考题 2: 为什么 DNN 加速不一定只看 retire 指令数?**

| 类别      | 名称       | 体会                           | 后续计划                       |
|---------|----------|------------------------------|----------------------------|
| 系统设计    | CPU/外设协同 | 硬件加速器不是孤立模块, 地址映射和一致性同样决定正确性 | 补充更完整的外设访问检查表              |
| RTL 编写  | 旁路策略     | MMIO 和共享 DDR 区域必须与普通缓存路径区分   | 将旁路范围参数化, 减少硬编码风险          |
| FPGA 调试 | 实现报告阅读   | 布局布线失败要结合拥塞等级、失败阶段和资源区域判断    | 记录不同配置下的 custom_bit_gen 日志 |

表 5: 收获总结

硬件加速会显著减少 CPU 执行的软件循环, 因此 retire 通常下降。但端到端性能还包括 MMIO 轮询、AXI 互联等待、DDR 访问、DNN 内部计算和 FPGA 时钟频率。如果硬件任务时间很长或系统实现拥塞严重, 即使 retire 减少, cycles 也可能没有达到预期。

## 五、 实验时长

在课后, 你花费了大约 7 小时完成此次实验, 其中各个阶段的用时如表 6 所示。

| 步骤 | 时长(小时) | 说明                                     |
|----|--------|----------------------------------------|
| 设计 | 2      | DNN 地址映射、MMIO 控制和一致性方案设计               |
| 开发 | 3      | 工程配置、D-cache 旁路和软件控制流程整理               |
| 调试 | 2      | dnn_test, custom_bit_gen 和 FPGA 运行日志分析 |

表 6: 实验时长